# 流水线 CPU 设计专题实验报告

**作者**：林岳　**学号**：2024K8009929018

**专题范围**：实践任务 7、8、9（五级流水 CPU 的建立、冒险处理与前递优化）

---

## 一、任务与判据

本专题从实践 6 的单周期 CPU 出发，做出一颗能正确跑通全部 20 个功能点（n1~n20）的五级流水 CPU，并按任务 7 → 8 → 9 的顺序逐级解决流水化带来的冒险。支持的指令为：`lu12i.w`、`add.w`、`addi.w`、`sub.w`、`slt`、`sltu`、`and`、`or`、`xor`、`nor`、`slli.w`、`srli.w`、`srai.w`、`ld.w`、`st.w`、`beq`、`bne`、`bl`、`jirl`、`b`。

验收判据是**逐指令比对**：每条指令写回时经调试接口上报，测试台与参考执行流逐条对照，不一致立即报错停下；n1~n20 全过时打印 `----PASS!!!`。上板对应的现象是数码管停在 `0x14000014`、双绿灯亮。

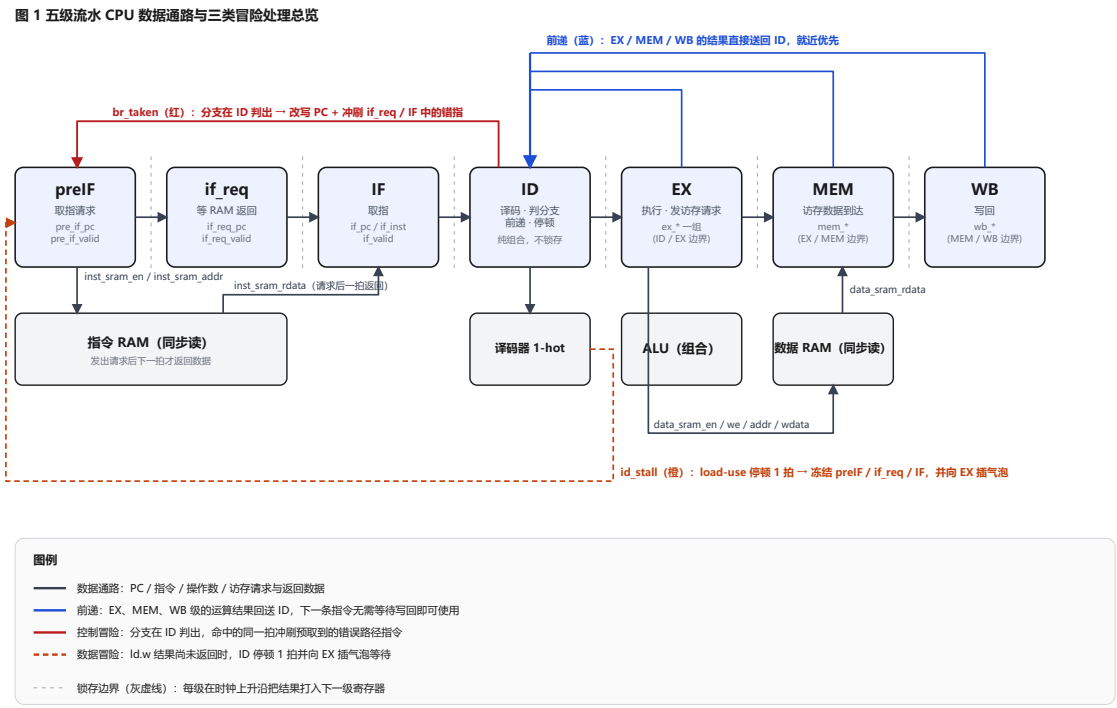
三个任务交付**三个逐级增量**的版本，每版只叠加一处机制，并附相对上一实验的 patch（可从单周期版逐级应用走到本版）：

| 任务 | 这一版新增的机制 | 测试镜像 |
|---|---|---|
| 7 | 五级流水骨架本身，**不处理冒险**（靠镜像在相关指令间插 4 条 nop） | 插 nop 版 |
| 8 | `cancel` 解控制冒险 + **阻塞**解数据冒险 | 无 nop 版 |
| 9 | 把阻塞换成 **EX / MEM / WB 三路前递**，只剩 load-use 停一拍 | 无 nop 版 |

## 二、设计思路

### 2.1 总体结构

整体结构如图 1 所示。流水线切成 5 个“干活级”——IF 取指、ID 译码、EX 执行、MEM 访存、WB 写回；由于**指令 RAM 是同步读**（本拍给地址、下一拍才回数据），取指一侧多插了 preIF（发请求）与 if_req（等返回）两个小级。这两级只是取指机制的“泵”，真正承担译码 / 执行 / 访存 / 写回职责的仍是标准的五级。




每一级之间用 `valid` / `ready_go` / `allowin` 三个信号握手：

```
allowin(本级) = !valid(本级) || ( ready_go(本级) && allowin(下一级) )
```

`allowin` 逐级向后依赖，天然形成一条从 WB 回传到 preIF 的**背压链**：任何一级停住，取指端自动跟着停，不需要额外的全局停顿信号。这条链是所有冒险处理的公共基础设施。

## 2.2 取指前端：同步 RAM 下 PC 与指令的配对

同步 RAM 与“理想存储器”最大的区别是**没有撤销键**：只要 `en=1`，下一拍必然返回数据，无法事后作废。由此有两个必须处理的问题。

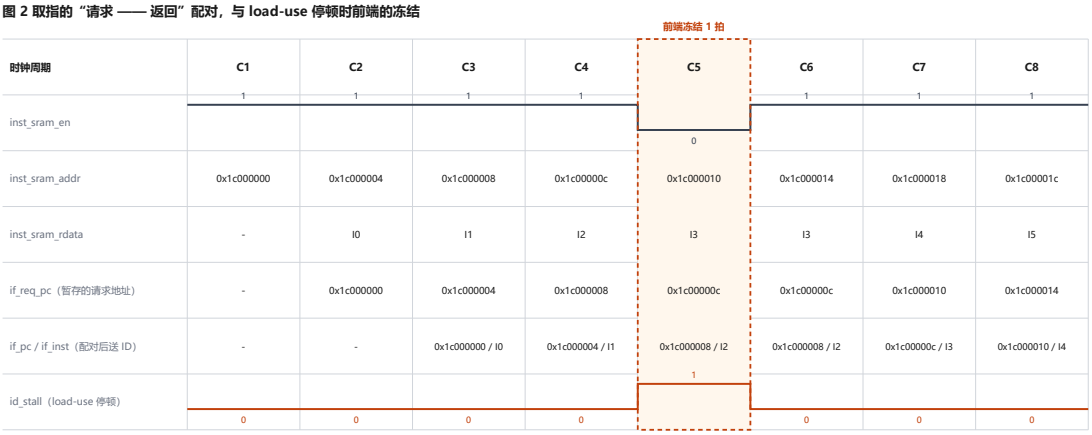
**其一，PC 与指令的对齐。** 请求与返回相隔一拍，只记 PC 会导致返回时分不清数据属于哪条指令。做法是加一级 `if_req`：暂存“已发出请求的 PC”，在数据返回的那一拍与读回数据**成对**锁入 IF。

**其二，停顿期间不能继续发请求。** 否则 RAM 照常返回数据，与暂存的 PC 错位，整条流水线随之错乱。做法是把请求使能与前端能否前进**绑死**（`en = pre_if_valid && pre_if_allowin`）：停顿发生时 `en=0`，地址不再前进、RAM 不再采样，**PC 与数据一起冻结**，配对关系不会被破坏。

一个容易写错的细节：`pre_if_pc` 的复位值取 `0x1bfffffc`（入口 `0x1c000000` 减 4），而不是入口本身。PC 只在时钟沿 +4，这样复位后第一次真正送上地址总线的值恰好是入口；直接填入口的“更合理”写法反而会跳过第一条指令。

图 2 是取指前端的时序：地址在 Ck 发出、数据在 Ck+1 返回；C5 发生 load-use 停顿时请求使能拉低，被跳过的请求在 C6 原样发出，配对始终成立。

图 2 取指的“请求 —— 返回”配对，与 load-use 停顿时前端的冻结




· 读法：地址在 Ck 发出，数据在 Ck+1 返回；if_req_pc 把“已发出请求的 PC”暂存一拍，返回时正好与 rdata 配对。
· C5 发生 load-use 停顿：id_stall=1 → pre_if_allowin=0 → inst_sram_en=0。此时地址与数据一起冻结，RAM 不会多读一条，配对关系不被破坏。
· C6 恢复：C5 被跳过的请求在 C6 原样发出，C3 请求的指令仍在 rdata 上，IF 在 C6 用 if_req_pc 与 rdata 完成配对后送 ID。

## 2.3 译码：独热译码与立即数拼接

LoongArch 指令按固定位段切分：先切字段，再把各字段送进独热译码器，最后把对应位段的独热线相与得到命中线，例如：

```
assign inst_add_w = op_31_26_d[6'h00] & op_25_22_d[4'h0] & op_21_20_d[2'h1] & op_19_15_d[5'h00];
```

选独热译码而不是 `case`，是因为每条指令的判定**互相独立**：加一条指令只是多加一行，综合器容易并行成比较树，也好读、不易写出优先级覆盖或漏 `default` 的错误。

立即数按 `need_ui5 / need_si12 / need_si16 / need_si20 / need_si26 / src2_is_4` 选择拼接：`jirl` / `bl` 取常数 4（算返回地址），`lu12i.w` 取 si20 左移 12 位，移位指令取 ui5 零扩展，其余按 si12 符号扩展。ALU 功能码 `alu_op` 同样用独热码，由各指令命中线按运算类别或起来。

## 2.4 三类冒险的处理

以下按任务 9 的最终版描述。任务 8 尚无前递（数据冒险一律用阻塞解决），任务 9 把阻塞换成三路前递；两版的控制冒险处理相同。

### 2.4.1 控制冒险：分支在 ID 判出，当拍冲刷

分支若等 EX 再判，错指已深入流水两级以上，要冲掉更多级。本设计把判定**提前到 ID**：`beq` / `bne` 用前递后的两个操作数直接比较，`b` / `bl` / `jirl` 直接算目标地址，全部在 ID 这一拍得到 `br_taken` 与 `br_target`。命中时同一拍把 PC 改写为目标，并用 `!br_taken` 把取指前端在途的两条错指同时作废（图 3）。代价是 2 个气泡，比推迟到 EX 再冲刷小得多。

**图 3 分支在 ID 判出：同一拍冲刷两条错指，代价是 2 个气泡**

br_taken 生效，两条错指同拍作废；C5、C6 各浪费 1 拍

| | C3 | C4 | C5 | C6 | C7 | C8 |
|---|---|---|---|---|---|---|
| preIF 发出的取指地址 | ~~错指1~~ | 错指2 (请求作废) | br 目标 | +4 | +8 | +0xc |
| if_req（等 RAM 返回） | i0 | ~~错指1 (作废)~~ | 空 | br 目标 | +4 | +8 |
| IF / ID（组合译码） | i-1 | beq (i0) → br_taken | 气泡 | 气泡 | br 目标 | +4 |
| EX | i-2 | i-1 | beq (i0) | 气泡 | 气泡 | br 目标 |
| MEM | i-3 | i-2 | i-1 | beq (i0) | 气泡 | 气泡 |
| WB | i-4 | i-3 | i-2 | i-1 | beq (i0) | 气泡 |

· ID 是纯组合级：它译码的就是 IF 本拍持有的指令，故两行同步；前递后的操作数让 beq 在 C4 的 ID 拍即可比较出 br_taken。
· 同一拍内：if_req 中在途的“错指1”被 !br_taken 清为无效，preIF 在 C4 已发出的“错指2”请求返回后被丢弃，两条错指都不会写回寄存器堆。
· 代价：C5、C6 两个周期没有有效指令交给 EX（2 个气泡），分支目标指令到 C7 才进 IF。若把分支推迟到 EX 再判，要冲刷的级数更多、代价更大。

### 2.4.2 前递：EX / MEM / WB 三路结果回送 ID

“上一条刚算完、下一条马上要用”时，等写回寄存器堆太晚。做法是把 EX、MEM、WB 三级的结果回送到 ID 的操作数选择上，按**就近优先**取最新的：EX 级给组合 ALU 结果，MEM 级给其保存的结果或本拍从数据 RAM 返回的 load 数据，WB 级给写回值。

三路都带过滤条件：`dest != 0`（x0 无意义）、`gr_we`（不写寄存器的指令不产生新值）；EX 那一路额外要求 `~res_from_mem`——EX 拍的 `ld.w` 只有地址、数据还没回来，前递过去会是错误的地址值。

实际选值就是一条四选一优先链：

```
wire [31:0] id_rj_value = fwd_ex_to_rj  ? ex_alu_result :
                          fwd_mem_to_rj ? mem_value     :
                          fwd_wb_to_rj  ? wb_result     :
                                          rf_rdata1     ;
```

优先级写反了在简单测试里未必暴露，但在“连续三条指令依赖同一寄存器”的序列里会立刻出错，因此这一处必须写死为 EX > MEM > WB > 寄存器堆。

### 2.4.3 load-use 停顿

`ld.w` 的数据到 MEM 级才从 RAM 出来，EX 拍无法前递。当“EX 正在做 load”且“ID 要用它写的那个寄存器”时置停顿：

```
wire id_stall = if_valid && ex_valid && ex_res_from_mem && ex_gr_we && (ex_dest != 5'b0)
             && ( (id_uses_rj  && (ex_dest == rj))
               || (id_uses_rkd && (ex_dest == (src_reg_is_rd ? rd : rk))) );
```

停顿一拍只冻结前端，EX 里的 `ld.w` 照常流到 MEM、向 EX 插一个气泡；下一拍数据已到 MEM，由 MEM→ID 前递供数，代价恰好 1 拍。判据里 `ex_gr_we`、`id_uses_rj / id_uses_rkd` 两个过滤排除“其实用不到”的情况——能少停一拍是一拍。

## 三、重点、难点与设计取舍

**最难的是同步 RAM 的“不可撤销”。** 解法（把请求使能与 `allowin` 绑死）只有一行代码，但要先想明白“同步读没有撤销键”，才写得出这一行。

**分支判在 ID 还是 EX。** ID 判的收益是只冲 2 级、只花 2 拍；代价是 ID 的组合路径变长（译码、前递选择、比较、目标计算都在一拍内完成）。本设计选 ID 判，依据是实测时序有充足余量，而冲刷是实打实的性能损失。

**前递的优先级次序与停顿过滤。** EX > MEM > WB 的次序不能含糊：写反了在简单测试里未必暴露，在“连续三条指令依赖同一寄存器”时会立刻出错。停顿判据则相反——从最保守的写法起步，不断用 `ex_dest != 0`、`ex_gr_we`、`id_uses_rj / id_uses_rkd` 这类过滤减少无谓停顿；这一类“正确但可以更聪明”的地方，是流水线设计里值得反复打磨的部分。

## 四、验证与结果

### 4.1 功能仿真

三个任务各跑一轮，均为 n1~n20 全过（仿真在 iverilog 与 Vivado xsim 两套流程下都复跑过）：

| 任务 | 测试镜像 | 结果（最后一点时刻） |
|---|---|---|
| 7（流水线建立） | 插 nop 版 | PASS，2058995 ns |
| 8（cancel + 阻塞） | 无 nop 版 | PASS，1349305 ns |
| 9（三路前递） | 无 nop 版 | PASS，639855 ns |

同一套测试程序下，结束时间本身就是阶段递进的证据：任务 8 比任务 7 快约 1.5 倍（镜像不再靠 4 条 nop 拉开距离），任务 9 又比任务 8 快约一倍（阻塞换成了前递）。

**反证**：把任务 7 的 RTL 原样放进任务 8 的镜像里运行，仿真在第 2077 ns 就报错——任务 7 的正确性依赖镜像里插好的 nop，三个版本的差别是实打实的。

### 4.2 综合、实现与上板

Vivado 2019.2，器件 xc7a200tfbg676-1。三个版本分别综合 / 实现 / 出 bit，均 0 Error、时序全部收敛：

| 版本 | 布线后时序 |
|---|---|
| 任务 7 | WNS = 3.922 ns，TNS = 0.000，WHS = 0.104 ns |
| 任务 8 | WNS = 2.499 ns，TNS = 0.000，WHS = 0.064 ns |
| 任务 9 | WNS = 2.112 ns，TNS = 0.000，WHS = 0.085 ns |

越简单的版本余量越大（任务 7 的 WNS 接近任务 9 的两倍），正对应“任务 8/9 用组合逻辑复杂度换流水线拍数”的取舍；余量最小的任务 9 仍有 2 ns 以上，说明把分支判定、三路前递都放进 ID 这一拍，对这颗器件与目标频率是安全的。

上板：三个版本的 bit 均在 2026-09 上板复验通过（`PROGRAM DONE`，数码管停在 `0x14000014`、双绿灯亮 = n1~n20 全过）。

## 五、调试记录

**案例 1：三个实验共用一份 RTL，增量补丁“几乎为空”。** 按“每版相对上一版做增量”自查时发现，7→8、8→9 的补丁几乎没有内容——开发时一口气写到底，任务 7 一版就把后两版的机制全写完了。对照实验：把任务 7 的 RTL 放进任务 8 的镜像，仿真立刻报错，证明三版不能合并。随后把 RTL 拆成三个真实增量，分别仿真、综合、复验。

**案例 2：仿真日志 20 秒写出 1.5 GB。** 日志全是逐周期读文件失败：测试台打开的参考执行流文件缺失，而比对循环还在跑，每拍报一次错。补齐文件后恢复正常，此后把“仿真前先确认参考执行流存在”固化为流程检查项。

**案例 3：同一条 RTL，命令行仿真器报“先用后声明”。** Vivado 工程模式编译通过、命令行却直接失败，核对报错行确认是代码里多处前向引用所致；为命令行流程加上与工程模式一致的宽松开关后通过。

**案例 4：GUI 里 `$finish` 之后仿真不退出。** 20 个测试点全过、`$finish` 已打印，但交互会话持续刷屏、直至卡死。把这次运行的日志与命令行跑出的日志逐字比对，内容完全一致——是 IDE 会话的收尾问题，不是设计问题。此后改用命令行流程（跑完自动退出），在 IDE 中则以看到 `----PASS!!!` 为准手动停止。

## 六、小结

1. 五级流水的骨架不难，难在**边界条件**：同步存储器没有撤销键、复位初值决定第一条指令取不取得到——这些不是“照着框图连线”能覆盖的。
2. `valid / ready_go / allowin` 握手是处理所有冒险的公共基础设施：控制冒险用它当拍作废错指，数据冒险用它冻结前端、插气泡；把这一套想清楚，之后加任何机制都是往同一个框架里填。
3. 三个任务的增量关系一句话：**7 先把流水线搭起来（冒险交给镜像里的 nop），8 用 cancel + 阻塞让它在不插 nop 的镜像上跑对，9 再把阻塞换成前递、把性能找回来。**最终版的思路是“能前递就不停，必须停就只停最小范围”：ALU 结果靠前递救回，只有 load-use 停一拍、且只冻前端。
4. 把分支判定提前到 ID，是“用组合逻辑复杂度换流水线拍数”的典型取舍；实测三个版本 WNS 分别为 3.922 / 2.499 / 2.112 ns，最复杂的任务 9 也留了 2 ns 以上余量。